

Relatório de Evolução do Projeto

EV Challenge — GoodWe · Sprint 03
Refractory conversacional em LangChain (LCEL)

Turma 1CCR · Grupo 1 — ver equipe completa na seção 5.

1. Resumo da evolução

Nas Sprints 1 e 2, o núcleo conversacional do chatbot ChargeGrid/EV Challenge GoodWe foi implementado de forma manual: uma classe ChargeGridChatbot chamando o Ollama (gemma2:2b) diretamente, com o system prompt embutido como string fixa no código Python (sem versionamento nem medição de tokens), memória de conversa feita à mão como uma lista de mensagens limitada a MAXIMO_HISTORICO=10 turnos, dados operacionais simulados hardcoded (SESSION_DATA) e sem validação estruturada da saída do modelo (resposta era texto livre, formatado por instrução de prompt). A avaliação oficial da Sprint 2 (84/100) documentou 7 casos de teste manuais (TC-01 a TC-07): 6 adequados e 1 reprovado (TC-06 — o chatbot recomendou a compra de um veículo em vez de recusar por estar fora de escopo).

Na Sprint 03, esse núcleo foi reconstruído sobre LangChain LCEL: uma chain declarativa (prompt | llm | parser), memória conversacional por sessão com limite de tokens, saída estruturada validada por um schema Pydantic v2 do domínio EV (ConsultaRecarga) e um system prompt versionado com context engineering leve (XML tagging). Guardrails determinísticos de segurança e escopo foram adicionados na frente da chain, bloqueando jailbreak/prompt injection e pedidos fora do domínio GoodWe (jurídico, financeiro, segurança elétrica) antes mesmo de o modelo ser chamado.

2. Refatoração — decisões técnicas e trade-offs

- Chain LCEL (prompt | llm | parser) substituindo chamadas manuais ao modelo: ganho em legibilidade e composabilidade, ao custo de uma curva de aprendizado inicial da sintaxe LCEL pela equipe.
- ConversationTokenBufferMemory encapsulada numa classe adaptadora (TokenLimitedHistory) para funcionar com RunnableWithMessageHistory: essa memória é uma API mais antiga do LangChain, não nativamente compatível com o padrão LCEL de histórico — o adaptador resolve isso sem abrir mão do controle por limite de tokens exigido no enunciado.

- Saída validada por Pydantic v2 (PydanticOutputParser): qualquer resposta que não bata com o schema ConsultaRecarga é rejeitada antes de chegar ao usuário, o que reduz drasticamente respostas malformadas em comparação com o parsing manual das Sprints 1/2.
- Guardrails determinísticos (regex) em vez de um segundo prompt de moderação: mais rápido e 100% auditável/testável sem depender do LLM, com o trade-off de exigir manutenção manual dos padrões conforme surgem novos tipos de ataque.

3. Comparativo antes/depois

Tabela obrigatória — evidência central do refactor. Métricas da Sprint 03 medidas em 2026-09-20 com evals/run_eval.py e docs/comparar_modelos.py, usando llama3.1:8b como substituto de gpt-oss:120b (indisponível localmente a tempo da entrega — ver docs/relatorio_modelos.md). Baseline das Sprints 1/2 extraído dos relatórios de avaliação oficiais do professor.

Métrica	Sprints 1/2 (manual/legado)	Sprint 03 (LCEL)
Qualidade das respostas (nota no eval)	6/7 casos adequados (85,7% — TC-06 reprovado)	11/11 casos aprovados (100%)
Tokens por turno (média)	não medido (prompt fixo no código, sem instrumentação)	834 tokens de system prompt (v3, medido com tiktoken) + até 800 de histórico (limite configurado)
Latência média por turno	não medido/reportado	~19,3 s (llama3.1:8b local, fim-a-fim incluindo guardrails — ver comparação de modelos)
Acurácia do structured output	N/A — saída em texto livre, sem schema	6/6 (100%) casos happy_path/edge_case validados pelo schema Pydantic v2
Casos de jailbreak bloqueados	1/1 (TC-07, Sprint 2)	2/2 (evals/sprint3_results.json)
Casos out-of-scope bloqueados	0/1 (TC-06 reprovado, Sprint 2)	3/3 (evals/sprint3_results.json)

4. Problemas encontrados e soluções

Incompatibilidade entre ConversationTokenBufferMemory e RunnableWithMessageHistory

ConversationTokenBufferMemory é uma API de memória mais antiga do LangChain, pensada para chains legadas — ela não implementa a interface BaseChatMessageHistory que RunnableWithMessageHistory espera. Decisão: criar uma classe adaptadora (TokenLimitedHistory) que expõe a interface esperada e delega a poda por limite de tokens para o ConversationTokenBufferMemory internamente. Mantém o requisito do enunciado (as duas ferramentas explicitamente pedidas) sem reescrever a lógica de poda do zero.

Formatos de retorno inconsistentes entre os dois guardrails

O guardrail de moderação (jailbreak) usa o campo `seguro` e o guardrail de escopo usa `permitido` para indicar se a mensagem passou. Ao combinar os dois num pipeline único (guarded_chain.py), isso gerava risco de checar o atributo errado silenciosamente. Decisão: normalizar a checagem com `getattr` e fallback explícito, documentando o motivo no código; fica registrado como possível refatoração futura (unificar num único dataclass de resultado) caso o grupo evolua os guardrails no Challenge.

Campo obrigatório do schema forçando o modelo a alucinar (potencia_kw)

O eval reexecutado (casos ec-01/ec-03) mostrou o modelo inventando um valor de potência mesmo sob a instrução do prompt v2 de "não inventar números", porque potencia_kw era um campo obrigatório do schema Pydantic — o parser rejeitaria qualquer resposta sem esse número, então o modelo era forçado a

estimar um valor plausível. Decisão: tornar potencia_kw opcional (default null) em src/schemas/consulta_recarga.py e atualizar o prompt (v3) para instruir explicitamente o uso de null nesse campo específico — mostrando que "não alucinar" via prompt sozinho não é suficiente quando o schema não dá essa saída ao modelo.

Modelo substituto respondendo em texto livre em vez de JSON válido

Com llama3.1:8b (substituto de gpt-oss:120b nesta entrega — ver docs/relatorio_modelos.md), alguns casos de borda (ex.: pergunta sobre potência negativa) faziam o modelo responder com uma explicação em prosa em vez do JSON do schema, quebrando o PydanticOutputParser com erro "Invalid json output". Decisão: ativar o modo JSON nativo do Ollama (ChatOllama(..., format="json")) em build_llm(), que força o modelo a sempre decodificar JSON sintaticamente válido. Isso não garante que os campos batam com o schema, mas eliminou 100% das falhas de parsing observadas no eval com esse modelo.

Caso de eval mal formulado (hp-03 testava memória de um dado nunca informado)

Ao reexecutar o eval das Sprints 1/2 sobre a versão LCEL, percebemos que o caso hp-03 ("quanto ficou o valor faturado?") esperava que o modelo recuperasse da memória um valor que o turno anterior (hp-02) nunca tinha mencionado — um teste estruturalmente impossível de passar. Decisão: corrigir a entrada de hp-02 em evals/eval_set.json para incluir o valor faturado, tornando hp-03 um teste real de memória de sessão (e documentando a correção no próprio arquivo). Os dois casos usam a mesma session_id em evals/run_eval.py para reproduzir a dependência entre os dois turnos.

5. Equipe e divisão de trabalho

Turma 1CCR · Grupo 1. Cada integrante liderou uma frente técnica mapeada diretamente a um bloco da rubrica, e todos colaboraram na consolidação final deste relatório.

Nome	RM	Tarefa principal
Léo Moreno Sambo	569556	Chain LCEL (prompt llm parser) e memória conversacional por sessão (RunnableWithMessageHistory + limite de tokens) — src/chain/builder.py, src/chain/memoria.py.
Fernando Hideki Rosa Oda	571408	Structured output — schema Pydantic v2 do domínio EV (ConsultaRecarga) e validações de campo — src/schemas/consulta_recarga.py.
Gabriel Botelho Romão	570589	System prompt versionado (v1 a v3) e context engineering (XML tagging, medição de tokens com tiktoken) — prompts/; coordenação técnica geral.
Thor Ferreira Camargo	569543	Guardrails de segurança — jailbreak/prompt injection e validação de escopo GoodWe (jurídico, financeiro, elétrico) — src/guardrails/.
Rafael Marinucci Peres	569729	Eval set e reexecução dos testes (happy path, edge case, jailbreak, out-of-scope) — evals/.
David dos Reis Cardoso	568938	Comparação de modelos (llama3.1:8b vs. gemma2:2b) e implementação do bônus multi-provider — docs/comparar_modelos.py, src/chain/multi_provider.py.